CS4475 Fall 2026
Amichai Fernandez
aaferanndez9@miners.utep.edu

Lab Report
Submitted on 10/24/26

# PA2: Priority-based Scheduler for xv6

## Github repo: https://github.com/amichaifernandez/xv6-riscv/tree/pa2

Please replace red text with your report text and any tables or figures, names of any accompanying files, etc. Remember to commit all the files for your lab submission, to put the URL for your private xv6 repo in the Teams assignment, to submit the Teams assignment, and to give the instructor and TA access to your repo.

Task 1. (15 pts) Implement the getprocs() system call and test it using the provided ps command.

Getprocs() was impelemented as a new system call. Sys_getprocs() in sysproc.c reads the addres of the user array with argaddr() and calls kgetprocs() in proc.c kgetprocs() loops through the process table whilie holding eait_lock and each process' lock, skips UNUSED and USED slots, fills in a struc pstat for each process, and copies to to the user arry with either_copyout(). I returns the number of prcesses

Changed files: kernel/pstat.h, proc.h, proc.c, sysproc.c, syscall.h, syscall.c, defs.h, user/user.h, user.usys.pl, user.ps.c, and the Makefile

AI helped me in error handling in the cases of ps.c uses enum procstate, which was only defined in kernel/proc.h, I moved it into pstat.h with a include guard and changed % to %ld in ps.c.

```
init: starting sh
$ ps
pid     state           size    ppid    name
1       sleeping        16384   0       init
2       sleeping        20480   1       sh
3       running         16384   2       ps
$ sleep 100 &
$ ps
pid     state           size    ppid    name
1       sleeping        16384   0       init
2       sleeping        20480   1       sh
6       running         16384   2       ps
5       sleeping        16384   1       sleep
$ QEMU: Terminated
```

Task 2. (15 pts) Modify the provided ps command to print the priority of each process.

A int priority field was added to struct proc. It's set to 0 in allocproc(), so the init starts at 0, and cpoed from the parent in kfork(), so the childn inherit it. Getpriority() returns the caller priority. Sertpriority(n) sets it to n if between 0 and "MAXPRIOIRTY" (49, defined in param.h) or returns 0

For getporocs() a prioirity field was added to the struct pstat and line line in the kgetprocs() to copy it. ps prints in the prio column

I had Claude Ai provide me with a test program prtest which checks the default value, setting and getting, rejecting 60 and -1, and inheritance. The ouput shows the child inherit 10 and changing to 20 without affecting the parent. Pexec 20 prtest shows inheritance through pexec

```
init: starting sh
$ prtest
start: priority = 0
after setpriority(10): priority = 10
setpriority(60) returned -1
setpriority(-1) returned -1
priority still = 10
child: priority = 10
child: after setpriority(20) = 20
pid     state           size    prio    ppid    name
1       sleeping        16384   0       0       init
2       sleeping        20480   0       1       sh
3       sleeping        16384   10      2       prtest
4       running         16384   20      3       ps
parent: priority = 10 (unchanged by child)
$ pexec 5 ps
pid     state           size    prio    ppid    name
1       sleeping        16384   0       0       init
2       sleeping        20480   0       1       sh
5       sleeping        16384   5       2       pexec
6       running         16384   5       5       ps
$ pexec 30 prtest
start: priority = 30
after setpriority(10): priority = 10
setpriority(60) returned -1
setpriority(-1) returned -1
priority still = 10
child: priority = 10
child: after setpriority(20) = 20
pid     state           size    prio    ppid    name
1       sleeping        16384   0       0       init
2       sleeping        20480   0       1       sh
7       sleeping        16384   30      2       pexec
8       sleeping        16384   10      7       prtest
9       running         16384   20      8       ps
parent: priority = 10 (unchanged by child)
$ $
```

Task 3. (15 pts) Add a readytime field to struct proc, initialize it correctly, and modify ps to print a process's age.

A readytime field was added to the struct proc. It's set tot eh current tick count at evey placne a process becomes RUNNABLE (uerinit, kfork, yield, wakeup, and kkill) getprocs() copies it into struct pstat

In ps, the age is uptime() – readtime if the process is RUNNABLE, and 0 otherwise, since age only means something it is waiting on the CPU. With round robin, 3 matmul processes showed ages 2, 1, and 0 which is the order in the ready queue; each waits at most 2 ticks.

```
$ matmul 10 &; matmul 10 &; matmul 10 &
$ ps
pid     state           size    age     prio    ppid    name
1       sleeping        16384   0       0       0       init
2       sleeping        20480   0       0       1       sh
9       running         16384   0       0       2       ps
8       runnable        16384   2       0       1       matmul
5       runnable        16384   1       0       1       matmul
7       runnable        16384   0       0       1       matmul
$ ps
pid     state           size    age     prio    ppid    name
1       sleeping        16384   0       0       0       init
2       sleeping        20480   0       0       1       sh
10      running         16384   0       0       2       ps
8       runnable        16384   2       0       1       matmul
5       runnable        16384   1       0       1       matmul
7       runnable        16384   0       0       1       matmul
$ CTime: 135 ticks
Time: 136 ticks
Time: 139 ticks
```

Task 4. (15 pts) Implement a priority-based scheduler.

SCHED_RR, SCHED_PRIO and SCHEDULER were added to param.h so the scheduler could be chosen at compile time. In scheduler() in procs, a priority scheduler that ruesn the RUNNABLE process with the highest priority was added, and the round robin was kept under the #else.

Claude AI helped me in writing schedtest.c, ehich starts 5 jobs with different priorities at onces, and that was added to the Make file as well

In schedtest 5000, the jobs finished in priority order and with round robin they finished all at the same tick.I learned that the scheduler is just a loop that picks which process runs next and that strict priority can starve low-priority processes.

Difficulties mainly consisting of errors based on mistyping. I also had to increase the number with my schedted (300) since it was too short to show priority, so I increased it to 5000

Priority scheduler:

```
init: starting sh
$ schedtest
pid 5  priority 20  finished at tick 1
pid 7  priority 0  finished at tick 2
pid 8  priority 15  finished at tick 3
pid 6  priority 10  finished at tick 4
pid 4  priority 5  finished at tick 4
all done after 4 ticks
$ schedtest 1000
pid 11  priority 20  finished at tick 6
pid 14  priority 15  finished at tick 8
pid 12  priority 10  finished at tick 9
pid 10  priority 5  finished at tick 12
pid 13  priority 0  finished at tick 13
all done after 13 ticks
$ schedtest 5000
pid 17  priority 20  finished at tick 17
pid 20  priority 15  finished at tick 29
pid 18  priority 10  finished at tick 41
pid 16  priority 5  finished at tick 54
pid 19  priority 0  finished at tick 66
all done after 66 ticks
$ QEMU: Terminated
```

Round robin:

```
init: starting sh
$ schedtest 5000
pid 4  priority 5  finished at tick 65
pid 5  priority 20  finished at tick 65
pid 6  priority 10  finished at tick 65
pid 7  priority 0  finished at tick 65
pid 8  priority 15  finished at tick 65
all done after 65 ticks
$
```

Task 5. (15 pts) Add aging to your priority-based scheduler.

The aging policy is effective priority – min(99, priority + (ticks – readytime) / AGERATE), with AGINGRATE = 1. The function effepriority() in proc.c computes it, and the scheduler uses that instead of the base priority. The longer a process waits, the higer its effective priority becomes. When it runs, the readytime resets, dropping back to its base priority. I added AGING, AGINGRATE and MAXEFFPRIOITIY to param.h

The test program created by calude, agetest, starts 5 jobs at the same time and reports each one's reponse and turn around time. Without aging the average reponse time was 26 ticks, and the priority-0 job waitited 52 ticks to start. With aging the average reponse droped to 10 ticks, and the priority 0 job stated at tick 19. Average tur around went from 39 to 42.9, and round robin was 62.6

I learned that aging prevents starvation while letting higher prioirty jobs run first.

Again most of my problems were typo based. In one instatnce the build was failing because effepriority() was unused, which AI helped me instructing me to wrap it in a #if.

```
init: starting sh
$ agetest
pid 5  priority 20  response 0  turnaround 13
pid 8  priority 15  response 13  turnaround 26
pid 6  priority 10  response 26  turnaround 39
pid 4  priority 5  response 39  turnaround 52
pid 7  priority 0  response 52  turnaround 65
average response 26.0 ticks, average turnaround 39.0 ticks
$ QEMU: Terminated
init: starting sh
$ agetest
pid 5  priority 20  response 0  turnaround 17
pid 8  priority 15  response 6  turnaround 33
pid 6  priority 10  response 11  turnaround 44
pid 4  priority 5  response 15  turnaround 55
pid 7  priority 0  response 18  turnaround 65
average response 10.0 ticks, average turnaround 42.8 ticks
$ QEMU: Terminated
init: starting sh
$ agetest
pid 5  priority 20  response 1  turnaround 61
pid 7  priority 0  response 3  turnaround 62
pid 4  priority 5  response 0  turnaround 63
pid 6  priority 10  response 2  turnaround 63
pid 8  priority 15  response 4  turnaround 64
average response 2.0 ticks, average turnaround 62.6 ticks
$ $
```

Task 6. (5 pts) Summary

I learned how the scheduler decides which process runs next, and how that choice affects turn around and response time. I also learned that strict porority can stave low priority processes, and that aging fixes this while still letting high priority jobs run first.

My main difficulties were typos, especially with the word “priority”!!! which was a problem for obvious reasons. These typos would stop the build and give errors. Claude was used for managing given errors, with having me grasp concepts, navigating github, and the previously mentioned tests for the code.

Extra Credit Task (10 points).

Explain how you implemented the getpriority and setpriority commands. Show how using these commands can improve the response time for interactive jobs.